UniRent Implementation Plan

Oct 14, 2026 · @Siddharth

How to use this plan

Everyone reads the shared sections first:

- Roles
- Architecture
- Repository layout
- Shared contracts
- Timeline

Then read only your own role section. Each role section lists the folders you own, what you build, a week-by-week checklist, and what you need from and give to others.

This plan implements the UniRent Product Plan. Where the two differ, the product plan wins on *what* the app does and this plan wins on *how*.

Tech choices marked (proposed) are listed again in the last section, so the team can confirm or swap them in week 1.

Final team roles

Three roles stay as they were and two are swapped: Siddharth now owns payments and security, and Sripranav now owns the mobile app. The payment core is now a plain Razorpay integration, so it is smaller than the PPT described. That lets Siddharth also take security for the whole app, which lightens the backend load on Rammohan. Sasvat picks up the event workers, which fill the middle weeks between pipeline setup and monitoring.

Person	Final role	Owns folders	Change from original
Sasvat S R	CI/CD, observability and event workers	`ci/`, `monitoring/`, `workers/`	Adds the Kafka workers: outbox relay, push notification sender, timers

Person	Final role	Owns folders	Change from original
Tata Naga Sripranav	Mobile app (UI/UX and frontend)	`frontend/`	Swapped with Siddharth (was the payment core). The app is now mobile (Expo) instead of the PPT's web frontend, and includes moderator and admin screens
Rammohan M	Core backend API	`backend/`	Same role. Sign-in, roles and payments move to Siddharth; the help chatbot is added
Siddharth S	Payments and security	`payments/`, `security/`	Swapped with Sripranav (was the frontend). Payment core narrowed to Razorpay, ledger, late fees, refunds and reconciliation. Takes sign-in, roles, authorization, rate limits, audit log and data masking
Parv Manyam	Environment, research and testing	`infra/`, `tests/`	Same role. Owns Docker Compose, Dockerfiles, seed data, cross-module test suites and the week-1 research tasks

Unit tests live with the code they test, so each owner writes their own. Parv's `tests/` folder holds tests that span modules: integration, end-to-end, security and performance.

Workload risk. The mobile app is the largest single piece. Moderator and admin screens sit in their own folder, `frontend/src/features/staff/`. If Sripranav is behind at the end of week 5, another member can take that folder over without touching the rest of the app.

Architecture and tech stack

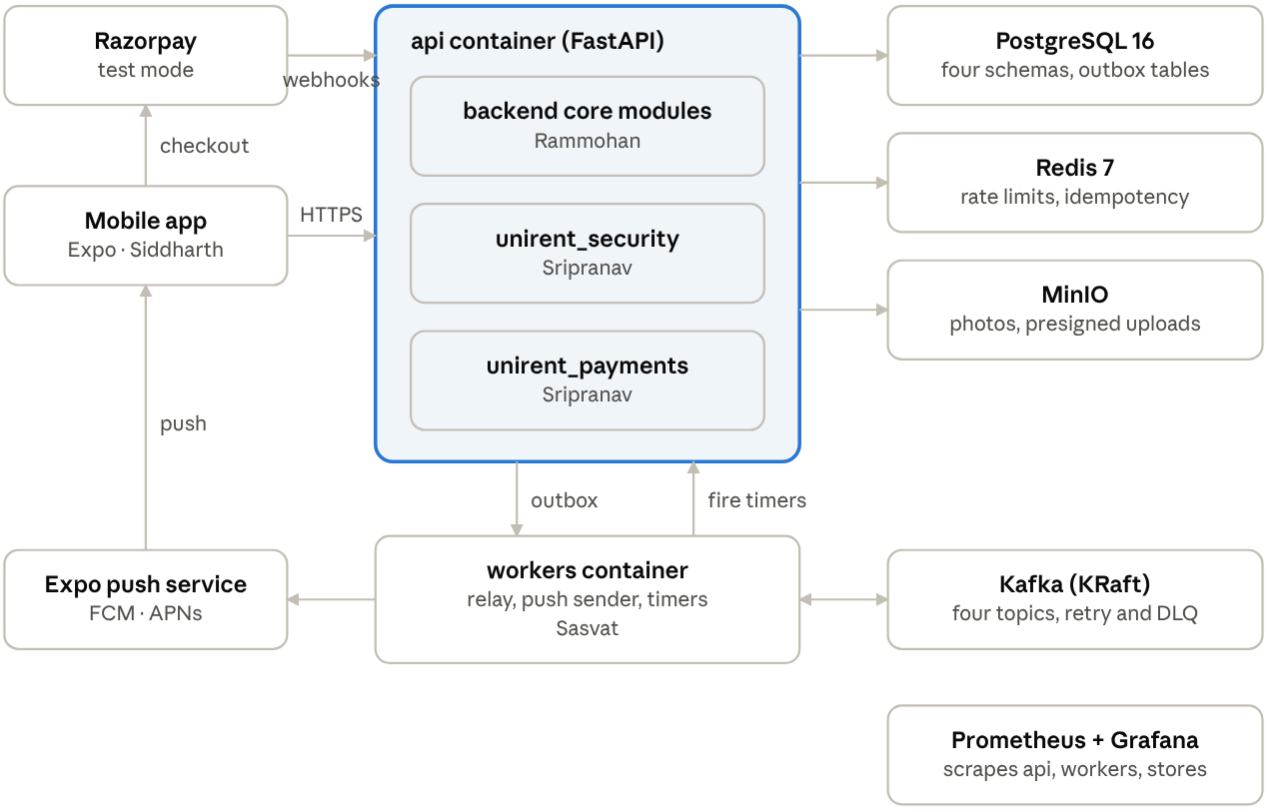
The system is a modular monolith. One FastAPI process serves the mobile app. Payments and security are Python packages installed into that process. Kafka carries side effects such as notifications and alerts, so a Kafka outage never blocks a handover or a payment.

Rules every module follows:

1. **Money and booking steps are synchronous.** Pickup, payment and OTP all happen inside API requests. Kafka never sits on that path.
2. **Events leave through an outbox.** Each event is written to an outbox table in the same database transaction as the change it describes. The workers' relay publishes it to Kafka. An event is never lost because Kafka was down, and never sent for a change that rolled back.
3. **Timers live in the backend, firing lives in workers.** The backend writes a timer row for every time-based rule:
   - request expiry
   - reminders
   - overdue alerts
   - escalation unlock
   - review release
   - stalled Proceed

   The workers' timer runner finds due rows and calls an internal API endpoint. The rule logic stays in one place.
4. **No cross-schema SQL.** A module reads another module's data only through that module's Python interface. The one exception is the workers' outbox relay, which reads the `outbox` tables directly.

One API container; side effects go through Kafka



runtime containers · dev and staging

The app talks only to the `api` container, plus Razorpay Checkout and direct photo uploads to MinIO. Everything asynchronous runs in the `workers` container.

Stack:

Layer	Choice	Owner
Mobile app	React Native with Expo, TypeScript, Expo Router. A development build, not Expo Go, because the Razorpay SDK needs native code	Sripranav
API	FastAPI on Python 3.12, one container `api`	Rammohan; Siddharth's packages mount into it
Database	PostgreSQL 16, one database, one schema per owner	Each schema's owner
Cache, rate limits, idempotency keys	Redis 7	Siddharth

Layer	Choice	Owner
Events	Kafka, single broker in KRaft mode, plus the outbox pattern	Sasvat
Workers	One `workers` container: outbox relay, push sender, timer runner, saved-search matcher, reconciliation trigger	Sasvat
Photo storage	MinIO (S3-compatible); the app uploads with presigned URLs (proposed)	Rammohan (API), Parv (container)
Student sign-in	Microsoft Entra ID, OAuth 2.0 with PKCE; the backend then issues its own JWTs	Siddharth
Staff sign-in	Email and password, hashed with Argon2	Siddharth
Phone OTP	Firebase Phone Authentication, with test numbers in development (proposed)	Siddharth
Payments	Razorpay test mode: Route linked accounts, Orders with transfers, Checkout SDK, webhooks, Refunds	Siddharth
Push notifications	Expo push service (FCM on Android, APNs on iOS)	Sasvat (sending), Sripranav (device registration)
Search	PostgreSQL full-text search, plus `pg_trgm` for type-ahead suggestions	Rammohan
CI/CD	Jenkins; images pushed to a registry; Docker Compose deploy to a staging host	Sasvat
Monitoring	Prometheus, Alertmanager, Grafana; Loki and Promtail for logs; exporters for Postgres, Redis, Kafka and containers	Sasvat
Environment	Docker Compose with `dev`, `test` and `staging` profiles	Parv

Layer	Choice	Owner
Testing	pytest and httpx (API), Locust (load), Jest and React Native Testing Library (app)	Each owner; Parv for cross-module suites

Repository layout, folder ownership and branching

Keep the existing `frontend/` and `backend/` folders and add nine more. Each person works on a branch and changes only the folders they own. The only shared folders are `contracts/` and `docs/`, and their edit rules are spelled out below.

```
unirent/
├── frontend/      Sripranav   Expo mobile app: student, moderator and admin
screens
├── backend/       Rammohan    FastAPI app, core modules, help chatbot
├── payments/      Siddharth   Razorpay, ledger, late fees, refunds,
reconciliation (Python package)
├── security/      Siddharth   Sign-in, tokens, roles, ownership checks, rate
limits, audit log (Python package)
├── workers/       Sasvat      Outbox relay, Kafka consumers, timer runner,
push sender
├── ci/            Sasvat      Jenkinsfile and pipeline scripts
├── monitoring/    Sasvat      Prometheus, Alertmanager, Grafana and Loki
configs
├── infra/         Parv        Compose files, Dockerfiles, env templates,
seed data
├── tests/         Parv        Integration, end-to-end, security and
performance suites
├── contracts/     Shared      OpenAPI spec and event schemas
├── docs/          Shared      Plans, decision records, runbooks
├── .github/CODEOWNERS
├── Makefile       Parv        make up, make test, make seed
└── README.md
```

Inside each folder:

Folder	Layout
`frontend/`	`app/` for Expo Router routes: `(auth)`, `(student)` tabs, `(staff)`. `src/features/<feature>/` for screens, hooks and state; `src/features/staff/` for moderator and admin screens. `src/api/` for the client generated from `contracts/openapi.json`. `src/ui/` for shared components and the theme
`backend/`	`app/main.py` mounts all routers. `app/core/` holds config, database, outbox writer and timer writer. `app/modules/<module>/` holds `router.py`, `service.py`, `models.py` and `schemas.py`. Also `migrations/`, `tests/` and `pyproject.toml`
`payments/`	Package `unirent_payments/` with `razorpay_client`, `orders`, `webhooks`, `ledger`, `late_fees`, `refunds`, `reconciliation` and `api` (FastAPI router). Also `migrations/` and `tests/`
`security/`	Package `unirent_security/` with `microsoft`, `staff_auth`, `tokens`, `phone_otp`, `roles`, `ownership`, `rate_limit`, `audit` and `masking`. Also `migrations/` and `tests/`
`workers/`	Package `unirent_workers/` with `relay`, `consumers/`, `timers`, `push` and `reconciliation_trigger`. Also `topics.yaml` and `tests/`
`ci/`	`Jenkinsfile` (Jenkins points at this path), `scripts/` for build, scan and deploy steps
`monitoring/`	`prometheus/` (scrape config, alert rules), `alertmanager/`, `grafana/` (provisioned dashboards as JSON), `loki/`, `promtail/`
`infra/`	`compose.yaml` with profiles, `docker/` (one Dockerfile each for api, workers and tests), `env/.env.example`, `seed/` for demo users, listings and bookings
`tests/`	`integration/`, `e2e/`, `security/`, `performance/` (Locust), `razorpay_sim/` (webhook simulator), `fixtures/`

Folder	Layout
`contracts/`	`openapi.json`, exported by CI from the running API, and `events/<EventName>.schema.json`

Branching and merge rules:

1. `main` is protected and always deployable; staging deploys from it.
2. Branch names follow `<name>/<topic>`, for example `parv/compose-kafka` or `siddharth/razorpay-orders`.
3. Change only folders you own. If you need something changed in someone else's folder, open an issue or ask them. For a small fix, open a PR and let them review it.
4. A PR merges only when Jenkins passes and the code owner of every touched folder approves. `.github/CODEOWNERS` enforces this.
5. Rebase on `main` at least once a day, and keep PRs small: aim for under 400 changed lines.
6. Each Python package has its own Alembic migration folder and its own database schema, so migrations from different owners never collide.
7. `contracts/`: whoever produces an API or event edits its file, and whoever consumes it reviews the change.
8. Root files (`README.md`, `CODEOWNERS`, `Makefile`, `.gitignore`, `.editorconfig`) are set up once in week 1 by Parv. After that, they change only by PR with team review.

Shared contracts

Everyone codes against four contracts:

- the HTTP API conventions
- the database schemas each person owns
- the in-process Python interfaces between packages
- the event catalog

Changing any of these needs a PR that every affected owner approves.

HTTP API conventions:

- **Paths:** everything is under `/api/v1`. Internal endpoints used by workers live under `/internal` and need a service token.
- **Formats:**
  - Money is an integer number of paise.

  - Timestamps are ISO 8601 in UTC and generated by the server; the client never sends deadlines.
  - IDs are UUIDs.
- **Errors:** always `{"error": {"code": "BOOKING_NOT_APPROVED", "message": "…"}}`, with a stable `code` the app can switch on.
- **Auth:** `Authorization: Bearer <access token>`.
  - Access tokens last 15 minutes. Refresh tokens last 30 days and rotate on each use.
  - Claims: `sub` (account ID), `role` (`student`, `moderator` or `admin`) and `status`.
- **Idempotency:** every request that moves money or changes a booking's state needs an `Idempotency-Key` header. Repeating a key within 24 hours returns the first response.
- **Pagination:** cursor-based: `?cursor=…&limit=…`.
- **Contract first:** Rammohan publishes stub endpoints for the whole API by the end of week 2. CI exports them to `contracts/openapi.json`, and Sripranav generates the app's typed client from that file.

Database schemas. One PostgreSQL database with one schema per owner. Tables in another person's schema are read only through their Python interface.

Schema	Owner	Main tables
`security`	Siddharth	accounts, staff_credentials, refresh_tokens, audit_log
`core`	Rammohan	profiles, assets, listings, listing_photos, bookings, booking_events, handover_photos, handover_otps, extensions, cancellations, reviews, trust_scores, saved_searches, appeals, support_threads, support_messages, faqs, device_tokens, timers, outbox
`payments`	Siddharth	linked_accounts, late_fee_rules, payments, ledger_entries, refunds, idempotency_records, reconciliation_runs, reconciliation_mismatches, outbox
`workers`	Sasvat	processed_events, dead_letters

In-process interfaces:

Package	Functions others call	Called by
`unirent_security`	`current_account` and `require_role(...)` (FastAPI dependencies), `require_owner(resource)`, `rate_limit(name)`, `audit.record(actor, action, target, details)`, `mask_serial(value)`, `set_account_status(account_id, status, by, reason)`, `hash_secret` and `verify_secret`	Backend, payments
`unirent_payments`	`payout_ready(account_id)`, `start_payment(booking_ref, purpose, payer, payee, amount_paise, method, idempotency_key)`, `mark_cash_paid(payment_id, by)`, `payment_status(booking_ref, purpose)`, `current_late_fee_rule()`, `late_fee(rent_paise, overdue_minutes, rule_id)`, `issue_refund(payment_id, amount_paise, ruling_ref)`	Backend
Internal API (backend and payments)	Backend: `GET /internal/timers/due`, `POST /internal/timers/{id}/fire`, `GET /internal/saved-searches/matches?listing_id=`, `GET /internal/accounts/{id}/push-tokens`, `DELETE /internal/push-tokens/{token}`. Payments: `POST /internal/reconciliation/run`	Workers

Event catalog. Each topic has matching `.retry` and `.dlq` topics.

Every event uses the same envelope: `event_id`, `type`, `version`, `occurred_at`, `producer`, `aggregate_id`, `payload`. Consumers skip any `event_id` they have already processed. Schemas live in `contracts/events/`.

Topic	Producer	Events	Consumed by
`bookings.events`	Backend	BookingRequested, BookingDecided (approved, declined, auto-declined, expired), BookingWithdrawn, BookingCancelled, PickupReminderDue, PickupCompleted, ExtensionRequested, ExtensionDecided, ReturnReminderDue, RentalOverdue, OverdueEscalationAvailable, ReturnCompleted, DamageChargeRecorded, ReviewPending, ReviewsRevealed	Push sender
`listings.events`	Backend	ListingPublished	Saved-search matcher
`moderation.events`	Backend	AppealFiled, AppealRuled, RulingOverruled, SupportReplyPosted, AccountStatusChanged	Push sender
`payments.events`	Payments	PaymentCaptured, PaymentFailed, CashMarkedPaid, RefundIssued, ReconciliationMismatchFound	Push sender; mismatches also raise an alert

Timeline and integration milestones

The 8 weeks follow the PPT's roadmap, but payments start in week 2 instead of week 5, because pickup can't be built without them.

Every week ends with an integration checkpoint: something the whole team can run on a phone against the shared environment. Features freeze at the end of week 7, and week 8 is for testing, fixes and demo rehearsal.

Week	Checkpoint the team demos to itself
1	`make up` brings every container up healthy on every laptop. Jenkins lints and unit-tests every PR. Research notes are in `docs/research/`.

Week	Checkpoint the team demos to itself
2	A student signs in with Microsoft, completes the profile and verifies their phone on a real device. Staff log in. Every API endpoint exists as a stub, and the app navigates all tabs on mock data.
3	An owner links a Razorpay test account and posts a listing with photos. Another student finds it with type-ahead search. An outbox event reaches a phone as a push notification.
4	Request → approve → pickup (photos, test UPI payment or cash, OTP) works end to end. Withdrawal, 24-hour expiry and cancellation with a note work.
5	Extension → overdue with late fees → return → Proceed works. Reminder and overdue pushes arrive. The integration suite covers the full rental.
6	Blind reviews, trust scores, damage charges, the sale flow and refunds work. The reconciliation report runs, and Grafana dashboards are live.
7	Appeals, rulings, admin overrule, escalations, the support inbox, the chatbot and FAQs, and the admin late-fee rule all work. The security suite passes. Staging deploys automatically from `main`. Feature freeze.
8	Load and resilience tests have run. Bugs are fixed, demo data is seeded, and the team has rehearsed the demo twice on staging.

Role: Sasvat S R — CI/CD, observability and event workers

You own three things:

- the Jenkins pipeline that gates every merge
- the monitoring that shows the system is healthy
- the workers that carry events, timers and push notifications

Owns: `ci/`, `monitoring/`, `workers/`.

Jenkins pipeline (`ci/`)

Run Jenkins in Docker on the staging host. A GitHub webhook triggers a build on every PR and every push to `main`. Secrets live in the Jenkins credentials store, never in the repo.

Stage	What it does	Fails the build when
1. Changes	Detects which folders changed	n/a
2. Lint	ruff and black for Python folders; eslint, prettier and `tsc` for `frontend/`	Any lint or type error
3. Unit tests	pytest in `backend/`, `payments/`, `security/`, `workers/`; Jest in `frontend/`; publishes coverage	Any failure
4. Integration	Starts Parv's `test` Compose profile, runs `tests/integration` and `tests/e2e`, tears down	Any failure
5. Security scan	pip-audit, npm audit, gitleaks for secrets, Trivy on built images	High or critical findings, or any secret
6. Contract check	Exports OpenAPI from the running API and compares it with `contracts/openapi.json`	The committed spec is out of date
7. Build and push	Builds `api` and `workers` images tagged with the commit SHA	Build error
8. Deploy (main only)	Pulls the new images on staging, runs migrations, restarts with the `staging` profile	Deploy error
9. Health check	Calls `/healthz` and `/readyz`, then a smoke test: staff login and listing search	Any check fails; the previous image tag is restored

From the PPT's merge gate: a build fails if any test under payments, security, bookings or reconciliation fails. Mark those suites so the pipeline can name them in the failure message.

On `main`, also trigger an EAS build of the Android app so testers always have the latest APK (proposed).

Monitoring (`monitoring/`)

- **Prometheus scrapes:**

  - `api` and `workers` at `/metrics`
  - postgres_exporter
  - redis_exporter
  - kafka-exporter, for consumer lag
  - cAdvisor
  - node_exporter
- **Metric names:** publish this list in week 5 so others can add the metrics in week 6. All names start with `unirent_`.

Metric	Type	Added by
`unirent_http_request_duration_seconds{route,method,status}`	Histogram	Rammohan (middleware)
`unirent_booking_transitions_total{from,to}`	Counter	Rammohan
`unirent_overdue_bookings`, `unirent_open_appeals`, `unirent_active_listings`	Gauge	Rammohan
`unirent_payments_total{method,outcome}`	Counter	Siddharth
`unirent_reconciliation_mismatches`	Gauge	Siddharth
`unirent_auth_failures_total{reason}`, `unirent_rate_limited_total{route}`	Counter	Siddharth
`unirent_events_processed_total{consumer,outcome}`, `unirent_event_retries_total`, `unirent_dlq_messages_total`	Counter	Sasvat
`unirent_push_sent_total{outcome}`	Counter	Sasvat

- **Grafana dashboards,** provisioned from JSON in the repo:
  - **API:** p50 and p95 latency and error rate per route.
  - **Payments:** success rate, UPI vs cash, failures, mismatches.
  - **Events:** consumer lag, retries, dead letters.
  - **Infrastructure:** database connections, Redis memory, container CPU and memory.
  - **Business:** active listings, bookings by state, overdue count, open appeals.
- **Alerts:**
  - API p95 above 1 second for 5 minutes
  - payment success below 90% over 15 minutes

    - consumer lag above 1,000 for 10 minutes
    - any dead-letter message
    - any reconciliation mismatch
    - any container down
- **Logs:** every Python service logs JSON lines with a `request_id`. Promtail ships them to Loki, so they're searchable in Grafana.

Workers (`workers/`)

- **Outbox relay:** polls `core.outbox` and `payments.outbox`, publishes each event to its topic keyed by `aggregate_id` so events for one booking stay in order, then marks the row sent.
- **Push sender:** consumes `bookings.events`, `moderation.events` and `payments.events`. Maps each event to its recipients and message text, following the Notifications table in the product plan. It fetches device tokens from the backend, sends through the Expo push API, checks receipts, and deletes dead tokens.
- **Saved-search matcher:** on `ListingPublished`, asks the backend which saved searches match, then pushes to each searcher.
- **Timer runner:** every 30 seconds, fetches due timers and fires each one. Firing is idempotent on the backend side, so a timer fired twice does nothing extra.
- **Reconciliation trigger:** calls payments' reconciliation run nightly at 02:00 IST.
- **Reliability:**
    - Failed messages retry with backoff three times, then go to `.retry`, then to `.dlq`.
    - `workers.processed_events` drops duplicates.
    - Workers shut down cleanly on SIGTERM.
- **Resilience experiments (week 8):** run each of these during load, and confirm no event is lost, no push is sent twice, and no money is affected:
    - kill the workers container
    - delay a consumer
    - stop Kafka for 5 minutes

    Write the results in `docs/runbooks/`.

Week by week

- [ ] W1: Jenkins running with the GitHub webhook; stages 1–3 pass for every folder; `CODEOWNERS` agreed with Parv.
- [ ] W2: Integration stage on Parv's `test` profile; contract check; image build and push.
- [ ] W3: Outbox relay and `topics.yaml`; push sender delivers a test event to a phone.

- [ ] W4: Push messages for request, decision, cancellation and pickup events; timer runner.
- [ ] W5: Reminder and overdue pushes; saved-search matcher; retries and dead letters. Publish the metric list.
- [ ] W6: Prometheus, exporters and all five dashboards; reconciliation trigger; security scan stage.
- [ ] W7: Staging deploy, health check and rollback; alerts; Loki logging.
- [ ] W8: Resilience experiments and runbooks; prepare the pipeline and dashboard part of the demo.

Interfaces

- **You need:**
  - Parv's `test` Compose profile (W2)
  - Rammohan's internal endpoints (W3–W5)
  - event schemas in `contracts/events/` (W3)
  - metrics from Rammohan and Siddharth (W6)
- **You give:**
  - CI results on every PR
  - push delivery for every event
  - timer firing
  - nightly reconciliation
  - dashboards and alerts

Done when:

- A PR with a failing payments test cannot merge.
- A push arrives within 10 seconds of the event.
- All five dashboards show live data on staging.
- Restarting the workers container loses no event.

Role: Tata Naga Sripranav — mobile app

You build the whole app: every screen in the product plan's screen map, for students, moderators and the admin. The backend decides all rules, so the app shows what the API returns and never works out deadlines, fees or visibility itself.

Owns: `frontend/`.

Setup

- Expo with TypeScript and Expo Router.
- Run it as a development build (`expo-dev-client`), not Expo Go, because `react-native-razorpay` needs native code.
- Build for Android first; iOS needs an Apple developer account (see decisions).

Libraries (proposed):

Need	Library
Server data and caching	TanStack Query
Session and UI state	Zustand
Forms and validation	react-hook-form with zod
Typed API client	openapi-typescript and openapi-fetch, generated from `contracts/openapi.json`
Microsoft sign-in	expo-auth-session (PKCE)
Token storage	expo-secure-store
Photos	expo-image-picker, expo-camera, expo-image-manipulator to resize before upload
Push	expo-notifications
Payment	react-native-razorpay
Mock API in weeks 1–2	MSW

Navigation

- `(auth)` : sign-in → profile form → phone OTP → terms → notification permission.
- `(student)` tabs: Home, Bookings, Listings, Profile. The booking screen is shared between Bookings and Listings.
- `(staff)` tabs, shown by role:
  - Moderators: Appeals, Support, Escalations, Users, Listings.
  - Admin: all of those, plus Reports, Late-fee rule and Moderators.
- Push notifications deep-link to the right screen, for example `unirent://bookings/{id}` .

Design system first (week 1)

Build these shared pieces in `src/ui/` before any feature screen:

- colour, type and spacing tokens
- buttons, inputs, cards
- trust badge and "New" badge
- star rating input
- status timeline
- checklist step
- countdown
- OTP display and OTP entry
- empty, loading and error states

From Zoomcar, borrow the flow, not the look: every booking screen shows a timeline of states and one card with the next action.

Behaviour rules

1. **Idempotency keys.** Generate an `Idempotency-Key` per user action and reuse it on retry. Never generate a new one for the same tap.
2. **Checklists enforce order.** In pickup and return checklists, each step unlocks only after the server confirms the previous one.
3. **Payments:**
   - **UPI:** ask the backend to start the payment, open Razorpay Checkout with the returned order, send the Razorpay IDs to the verify endpoint, then refetch the booking.
   - **Cash:** the renter taps "Pay with cash", and the owner sees "Mark cash received".
4. **OTP direction.** At pickup the renter's phone shows the code and the owner types it. At return it is the other way round.
5. **Photos.** Resize to at most 1600 px, upload to the presigned URL, then confirm with the API.
6. **Reviews.** The review screen can't be left until it is submitted, then waits for the other side or for release.
7. **Times and money come from the server.** That includes the overdue banner's live late fee. Show times in IST.
8. **Refetch on push.** When a push arrives for an open booking, refetch it so the screen updates without a pull-to-refresh.

9. **Help.** Show the FAQs list and a chat thread. Each bot reply is either an answer or a fallback with "See FAQs" and "Talk to a moderator" buttons. Moderator replies appear in the same thread.

Week by week

- [ ] W1: Expo project and dev build running on an Android phone; design system; navigation skeleton with every tab.
- [ ] W2: Microsoft sign-in, profile form, phone OTP, terms, push permission and token registration; generated API client; screens on mock data.
- [ ] W3: Home with type-ahead search, filters and sort; listing detail; add and edit listing with photo upload; payout account linking; saved searches.
- [ ] W4: Request flow (date picker, summary); incoming requests; booking screen with timeline; withdraw and cancel with note; pickup checklist with Razorpay, cash and OTP.
- [ ] W5: Active rental with countdown and extension; overdue banner; return checklist with damage charge and Proceed.
- [ ] W6: Blind compulsory reviews; own and public profiles with badges and reviews; payment history; sale flow.
- [ ] W7: Help with FAQs, chatbot and moderator handoff; appeal form; all staff screens.
- [ ] W8: Polish, accessibility pass (text size, contrast, labels), bug fixes, demo build.

Interfaces

- **You need:**
  - API stubs and `contracts/openapi.json` from Rammohan (W2)
  - Microsoft app registration details from Siddharth (W2)
  - Razorpay test keys and the payment start and verify endpoints from Siddharth (W4)
  - push messages from Sasvat (W3 onwards)
- **You give:**
  - Android builds for testing
  - early feedback on any API shape that makes a screen awkward

Done when:

- Every screen in the product plan's screen map exists.
- The full demo path runs on two Android phones without developer tools.

Role: Rammohan M — core backend API

You own the FastAPI app and every product rule that isn't money or security. That covers profiles, listings, search, the booking state machine, reviews and trust, moderation and the chatbot. The app trusts your API completely, so every rule in the product plan is enforced here.

Owns: `backend/`.

Foundations (`app/core/`)

- **Configuration and database:** settings with pydantic-settings; SQLAlchemy 2.0 (async) with asyncpg; Alembic for the `core` schema.
- **App wiring:** `main.py` mounts your routers plus Siddharth's `unirent_payments.api` and `unirent_security` routers.
- **Middleware:** request ID, JSON logging, the error format, and Prometheus metrics.
- **Outbox writer:** `emit(session, type, aggregate_id, payload)` writes to `core.outbox` in the caller's transaction.
- **Timer writer:** `schedule(session, kind, ref_id, due_at)` and `cancel_timers(ref_id, kind)`.
- **Clock:** `now()`. In staging only, admins can set a clock offset so the demo can jump to an overdue rental (proposed).
- **Health:** `/healthz` (process up) and `/readyz` (database and Redis reachable).

Modules (`app/modules/`)

1. Profiles

- Create the profile on first sign-in, against the account from `unirent_security`.
- Onboarding fields: photo, name, phone, roll number, department, year, and residence (hostel and room, or address).
- Every field is editable except the college email.
- Visibility is applied in the response models, never by the app:
  - public view
  - counterparty view after approval, which adds residence
  - staff view, which adds email and roll number

2. Listings and assets

- An asset is the persistent item record: owner, manufacture date, serial number, condition, and history. A listing is one offer of that asset, for rent or for sale.
- Photos upload to MinIO through presigned URLs; the app then confirms each upload.

- Before a listing is published, check `payout_ready(owner)`, then emit `ListingPublished`.
- Refuse edits while a booking is approved or active.
- Serial numbers go out through `mask_serial` in public responses.
- On a completed sale, move the asset, with its history, to the buyer.

3. Search and saved searches

- A generated `tsvector` covers title, description and category, for full-text search.
- `GET /search/suggest?q=` uses `pg_trgm` for type-ahead.
- Filters: category, type, price range, available dates.
- Sorts: trust, newest, price.
- With trust sort, an owner without a score yet ranks at the campus median.
- Saved searches: create, list, delete. The internal match endpoint serves the workers.

4. Bookings: the state machine. This is the heart of the backend. Rentals and sales share it, but sales skip Active, Overdue and Return.

From	To	Who triggers it
Requested	Approved, Declined, Withdrawn, Expired	Owner; requester; 24-hour timer
Approved	Active (rent) or Awaiting reviews (sale; the 48-hour appeal window opens)	Pickup checklist finished with OTP
Approved	Cancelled	Either side, with a note
Active	Overdue	Timer at end time plus grace period
Active or Overdue	Returning	Renter starts the return
Returning	In appeal	Either side disputes a damage charge
Returning	Awaiting reviews	Owner taps Proceed after charges are paid
In appeal	Awaiting reviews	Moderator ruling
Awaiting reviews	Completed	Both reviews in, or 24-hour release timer

Rules for every transition:

- Lock the booking row (`SELECT … FOR UPDATE`).
- Check that the actor is allowed.
- Write a `booking_events` row (append only) and an outbox event in the same transaction.
- Accept an `Idempotency-Key`.

Two approved bookings for the same listing can never overlap. A Postgres exclusion constraint on (listing, time range) enforces this for approved and later states, which settles the "concurrent reservation" test from the PPT.

- **Pickup and return checklists:**
  - Photo uploads are tracked per side.
  - Payment status comes from `payment_status()`.
  - The handover OTP is 6 digits, generated by the server once photos and payment are done. It is stored hashed and valid for 15 minutes, with 5 attempts allowed (using `rate_limit`).
- **Late fees:** lock the late-fee rule ID at approval, then compute the live fee with `late_fee()`.
- **Extensions:** new end time × listing rate. Start the payment with purpose `extension`, move the end time, and reschedule timers.
- **Damage charges:** the owner records an amount. The renter accepts by paying, or either side appeals.
- **Timers to schedule:**
  - request expiry (+24 h)
  - pickup reminder (−1 h)
  - return reminder (−1 h)
  - overdue start
  - hourly overdue alerts
  - escalation unlock (+1 h overdue)
  - stalled Proceed (+24 h after return OTP)
  - review release (+24 h)
  - sale appeal window close (+48 h)

5. Reviews and trust

- Reviews are stored hidden and revealed when both sides submit or the release timer fires.

- Parameters depend on the review type, as listed in the product plan. Stars run 1–5 and a comment is required.
- A cancellation rating is one star score plus a comment.
- Recompute trust when reviews are revealed, a return completes, a booking is cancelled, or a ruling lands.

Trust formula (proposed; weights live in config):

```
Lender trust = 100 × (0.70·R + 0.15·C + 0.15·A)
Renter trust = 100 × (0.50·R + 0.20·P + 0.10·D + 0.10·C + 0.10·A)

R = (average overall rating of the last 20 reviews in that role − 1) ÷ 4
P = average over rentals of max(0, 1 − hours late ÷ 24)
D = 1 − min(1, upheld damage charges ÷ rentals)
C = 1 − min(1, 2 × cancellations ÷ trades)
A = min(1, account age in days ÷ 180)

Subtract any moderator penalty, clamp to 0–100.
Show nothing until the user has 5 completed trades.
```

6. Moderation

- **Appeals:** created with handover photos attached automatically. Rulings record charge, refund, trust penalty and account action. Refunds go through `issue_refund()`. The admin can overrule a ruling.
- **Escalations queue:** overdue by 1 hour or more, and stalled Proceeds.
- **Listings:** removal.
- **Users:** suspend and ban through `unirent_security`; trust penalties; review removal.
- **Risk flags (proposed), each within 30 days unless stated:**
  - 3 or more overdue returns
  - 3 or more cancellations
  - 2 or more disputes
  - trust below 30
- **Support threads:** created by the chatbot handoff; moderators reply into them.
- Every staff action calls `audit.record()`.

7. Help chatbot

- A `faqs` table, seeded from the product plan's FAQ list, holds each question, answer and keywords.
- The matcher is deterministic, with no AI model:
  1. Normalise the message.

2. Score it against each FAQ by keyword overlap and trigram similarity.
3. Answer if the best score clears a threshold. Otherwise reply with the fallback and two actions: "See FAQs" and "Talk to a moderator".

- Handoff creates a support thread holding the transcript.

8. Devices and staff metrics

- Register and remove push tokens.
- Internal endpoints for the workers.
- `GET /admin/metrics` for the admin's platform metrics screen.

Week by week

- [ ] W1: FastAPI skeleton; `app/core` (database, settings, outbox, timers, errors, health); Alembic for `core`.
- [ ] W2: Stub every endpoint with request and response models, and export `openapi.json`. Profiles and onboarding on Siddharth's accounts; push token registration.
- [ ] W3: Assets, listings and photos; search with suggestions, filters and sorts; saved searches; `ListingPublished`; internal saved-search endpoint.
- [ ] W4: Booking state machine: request, approve with exclusion constraint, decline, withdraw, expiry, cancel with note; pickup checklist with payment and OTP.
- [ ] W5: Active, extension, overdue with late fee, return, damage charge, Proceed; all timers and internal timer endpoints.
- [ ] W6: Reviews and trust; sale flow and asset transfer; metrics from the metric list.
- [ ] W7: Moderation, admin overrule, risk flags, support threads, chatbot, admin metrics, staging clock.
- [ ] W8: Fixes, indexes and query plans, demo seed data with Parv.

Interfaces

- **You need:**
  - `current_account`, `require_role`, `require_owner`, `rate_limit`, `audit`, `mask_serial` and account-status functions from Siddharth (W2)
  - payment functions from Siddharth (W4)
  - the Compose environment from Parv (W1)
- **You give:**
  - `contracts/openapi.json` (W2)
  - event schemas for `bookings`, `listings` and `moderation` (W3)

- internal endpoints for workers (W3–W5)

Done when:

- Every booking transition has a unit test for each allowed and each forbidden actor.
- The overlapping-approval test passes under concurrency.
- Every rule in the product plan's business rules table is enforced by the API.

Role: Siddharth S — payments and security

You own two Python packages that every other module depends on:

- `unirent_security`: who the user is and what they may do
- `unirent_payments`: every rupee that moves, and the proof that it moved exactly once

Owns: `security/`, `payments/`.

Security (`security/unirent_security/`)

- **Microsoft sign-in.** Register an app in Microsoft Entra ID.
  1. The mobile app does the OAuth flow with PKCE and sends the ID token to `POST /auth/microsoft`.
  2. The backend checks the token's signature against Microsoft's published keys, plus its audience, issuer, tenant ID and email domain.
  3. It finds or creates the account and issues UniRent tokens.
- **Staff sign-in.**
  - `POST /auth/staff/login` with Argon2id password hashes.
  - Lock the account for 15 minutes after 5 failed attempts.
  - The admin creates moderators with `POST /admin/moderators`, and new moderators set their own password on first login.
- **Tokens.**
  - Access tokens are JWTs that last 15 minutes.
  - Refresh tokens are random strings stored hashed, rotated on each use. Reusing an old refresh token revokes the whole chain.
- **Phone verification (proposed).** The app verifies the number with Firebase Phone Authentication. The backend checks the Firebase ID token and saves the verified number. Changing the number needs verification again.
- **Roles and ownership.** Ownership checks are the defence against broken object-level authorization, the first item in the PPT's security tests.
  - `require_role(...)` checks the role.

    - `require_owner(...)` checks the object, for example:
        - only the renter, the owner and staff can read a booking
        - only the owner can edit a listing
        - residence details show only to the counterparty after approval
- **Account status.** `set_account_status()`:
    - **Suspended:** blocks new listings, requests and payments, but allows finishing open handovers.
    - **Banned:** blocks sign-in and revokes all tokens.
- **Rate limits.** Redis, sliding window:

Route	Limit
Student sign-in	10 per minute per IP
Staff login	5 per 15 minutes per account
Handover OTP entry	5 attempts per code
Payment start	10 per minute per user
Chatbot	20 messages per minute per user
Everything else	120 per minute per user

- **Audit log.**
    - Append only. The app's database user has INSERT but no UPDATE or DELETE on `security.audit_log`.
    - Record every sign-in, staff action, ruling and payment action.
- **Data protection.**
    - `mask_serial()` shows only the last 4 characters.
    - A logging filter strips phone numbers, emails, residence details and tokens from logs.
    - Secrets come only from environment variables.

Payments (`payments/unirent_payments/`)

- **Payout accounts.**
    - Create a Razorpay Route linked account for each owner and seller. Bank or UPI details go straight to Razorpay.
    - Store only the Razorpay account ID and its status. `payout_ready(account_id)` returns whether listing is allowed.

- **Payment purposes:** `rent`, `extension`, `return_charges` (late fee plus damage charge, itemised), `sale`.
- **UPI flow:**
  1. `start_payment()` creates a Razorpay Order. The order carries a transfer of the full amount to the payee's linked account.
  2. The app opens Checkout.
  3. `POST /payments/{id}/verify` checks the checkout signature.
  4. `POST /payments/webhooks/razorpay` checks `X-Razorpay-Signature` and handles these events:
     - `payment.captured`
     - `payment.failed`
     - `transfer.processed`
     - `refund.processed`

  Verify and webhook may arrive in either order. Whichever comes first moves the payment, and the second does nothing.
- **Cash flow.** `start_payment(method="cash")` creates a pending payment, and only the payee can call `mark_cash_paid()`.
- **Payment states:**

From	To	On
Created	Pending	Order created, or cash chosen
Pending	Captured	Verified checkout or captured webhook
Pending	Failed	Failed webhook; the user can retry with a new payment
Pending	Marked paid	Payee marks cash received
Captured or Marked paid	Refunded or Partly refunded	Refund issued after a ruling

- **Ledger.**
  - Double-entry and append only. Every captured, cash-marked or refunded payment writes balancing debit and credit lines, linked to the payment and booking.
  - Corrections are new lines.
  - The database user can't UPDATE or DELETE ledger rows.
  - A test asserts that debits always equal credits.
- **Idempotency.**

  - `Idempotency-Key` is checked in Redis first, then in `idempotency_records`, which stores a hash of the request and the saved response.
  - The same key with a different request body returns 422.
- **Late-fee rule.**
  - `late_fee_rules` is versioned: grace minutes, base fee, hourly percentage, effective from, and who set it.
  - Admin endpoints read the rule and create a new version.
  - Formula: zero within the grace period; after it, base fee + rent × hourly percentage × full hours past the grace period, rounded to the nearest rupee.
  - Example: ₹50 rent, ₹10 base, 2%, 3 hours late → ₹10 + ₹3 = ₹13.
- **Refunds.**
  - `issue_refund()` refunds through Razorpay and reverses the transfer from the linked account.
  - For a cash payment, it records a cash refund that the owner marks as done.
- **Reconciliation.** Nightly, and on demand from the admin screen:
  1. Pull the day's Razorpay payments, transfers and settlements.
  2. Compare them with the ledger. A mismatch is any of: missing on either side, a different amount, or a different status.
  3. Write mismatches, emit `ReconciliationMismatchFound`, and serve the report to the admin.
- **Payment history** endpoint for each user.

Merge-gate tests you own

- The same idempotency key twice creates one payment.
- A replayed webhook does nothing.
- Verify and webhook racing in both orders give one capture.
- A bad webhook signature is rejected.
- The ledger always balances.
- A table of late-fee cases.
- A refund reverses the transfer.
- Token reuse revokes the chain.
- Every role and ownership helper allows and denies correctly.

Week by week

- [ ] W1: Package skeletons; Alembic for the `security` and `payments` schemas; Razorpay Route test-mode spike with Parv.

- [ ] W2: Microsoft sign-in end to end, tokens, staff login, roles and ownership helpers, rate limits, phone verification, audit log. Publish the security functions.
- [ ] W3: Payout accounts and `payout_ready`; payments model and states; idempotency; ledger.
- [ ] W4: UPI with Orders and transfers, verify, webhooks; cash marking. Pickup works end to end.
- [ ] W5: Versioned late-fee rule and calculator; extension and return-charge payments.
- [ ] W6: Refunds; reconciliation and report; payment history; payment and auth metrics.
- [ ] W7: Account status; review ownership checks on every endpoint against Parv's security suite; log masking; admin late-fee API.
- [ ] W8: Fix security findings; prepare the reconciliation part of the demo.

Interfaces

- **You need:**
  - calls from Rammohan's booking flows (W4)
  - Parv's Razorpay webhook simulator (W4)
  - Sasvat's nightly trigger (W6)
- **You give:**
  - security functions to everyone (W2)
  - payment functions to Rammohan (W3–W4)
  - `payments.events` schemas (W3)
  - Razorpay test keys and checkout details to Sripranav (W4)

Done when:

- No test can produce two charges for one action.
- The ledger balances after every test run.
- Every endpoint has an authorization test.
- Reconciliation reports zero mismatches on seeded demo data.

Role: Parv Manyam — environment, research and testing

You make sure everyone can run the whole system with one command. You answer the risky unknowns in week 1. And you build the test suites that prove the modules work together, including every test listed in the PPT.

Owns: `infra/`, `tests/`, and the root files (`Makefile`, first version of `CODEOWNERS`, `.gitignore`, `.editorconfig`, `README.md`).

Research (week 1, notes in `docs/research/`)

1. **Microsoft sign-in with the college tenant.** Can students consent to a new app themselves, or must college IT grant admin consent? Find the tenant ID and email domain. Find a fallback if consent is blocked, such as a test tenant plus the domain check.
2. **Razorpay Route in test mode.**
   - linked account creation
   - Orders with transfers
   - refunds with transfer reversal
   - webhooks to a laptop through a tunnel
   - whether the settlements API returns data in test mode
3. **Phone OTP.** Firebase Phone Authentication test numbers and quotas, compared with Indian SMS providers, which need DLT registration.
4. **Expo.** A development build with `react-native-razorpay`; Expo push; what iOS testing needs.
5. **Staging host.** Options and cost: a college server, a free-tier cloud VM, or one team laptop.

Docker environment (`infra/`)

Service	Notes	Profiles
`api`	Built from `infra/docker/api.Dockerfile` with the repo root as context. Installs `security/` and `payments/` as packages. Hot reload in `dev`	dev, test, staging
`workers`	Built from `infra/docker/workers.Dockerfile`	dev, test, staging
`postgres`	PostgreSQL 16 with `pg_trgm`. An init script creates the four schemas and the app's database user, with no UPDATE or DELETE on the ledger and audit tables	all
`redis`	Redis 7	all
`kafka`	Single broker in KRaft mode, plus an init job that creates topics from `workers/topics.yaml`	all
`minio`	Plus an init job that creates the photos bucket	all
`kafka-ui`	For browsing topics while developing	dev

Service	Notes	Profiles
Monitoring stack	Prometheus, Alertmanager, Grafana, Loki, Promtail, exporters, cAdvisor, using Sasvat's configs from `monitoring/`	dev (optional), staging
`razorpay-sim`	Your webhook simulator from `tests/razorpay_sim/`	test
`test-runner`	Runs the suites in `tests/`	test
`tunnel`	cloudflared, so Razorpay test webhooks reach a laptop (proposed)	dev

- Every service has a healthcheck, and dependents wait for `healthy`.
- `infra/env/.env.example` lists every variable and who owns it. Real secrets are never committed.
- **Makefile targets:** `up`, `down`, `logs`, `migrate`, `seed`, `reset`, `test-unit`, `test-integration`, `test-e2e`, `test-security`, `test-load`.
- **Seed data** (`infra/seed/`):
  - 10 students, 2 moderators and 1 admin
  - 20 listings across all categories
  - at least one booking in every state
  - a late-fee rule
  - linked test payout accounts

Test suites (`tests/`)

Each test sets up its own data and cleans up after itself. Suites run against the `test` Compose profile, with real Postgres, Redis and Kafka, and the Razorpay simulator sending properly signed webhooks.

Every test from the PPT, and where it lives:

PPT test	Suite	Notes
Duplicate payment request creates one effect	integration	Same key sent concurrently and sequentially
Concurrent reservation of the same asset	integration	Parallel approvals of overlapping requests; exactly one wins
Full payment and rental lifecycle	e2e	Request to Completed, UPI and cash variants

PPT test	Suite	Notes
Webhook retry and signature verification	integration	Replays, out-of-order delivery, bad signatures
Payout release and partial release	integration	Now covered by transfers to linked accounts and partial refunds after a ruling; there are no deposits
Broken object-level authorization	security	Every ID-taking endpoint called with another user's IDs
Privilege escalation on admin and settlement APIs	security	Student and moderator tokens on admin routes; tampered role claims
Rate-limit bypass on auth and payment routes	security	Header spoofing, token rotation, parallel clients
Serial numbers masked in public listing APIs	security	Every public response scanned for full serials and residence data
Dependency and secret scanning	CI	Owned by Sasvat
Burst notification fan-out	performance	Many events at once; push sender throughput and lag
Payment retry storms	performance	Repeated payment starts and webhooks under load
Kafka consumer lag under load	performance	Run with Sasvat while watching the dashboards
Database contention on hot listings	performance	Many students requesting one listing
Delayed settlement processing	resilience	Delayed webhooks and settlements; reconciliation must still match
State transitions, idempotency, ledger maths, trust	unit	Owned by Rammohan and Siddharth in their folders

Also add e2e tests for:

- the sale flow
- cancellation with a note and the rating that follows
- extension
- overdue with late fees, using the staging clock
- appeal, ruling and admin overrule

- blind reviews and the 24-hour release
- reconciliation showing zero mismatches

File bugs as GitHub issues labelled with the owning folder.

Week by week

- [ ] W1: Research notes; root files and `CODEOWNERS`; Compose with Postgres, Redis, Kafka, MinIO and placeholder api and workers; Makefile.
- [ ] W2: `test` profile and test runner image; database users and grants; first seed data; integration test harness; tunnel.
- [ ] W3: Razorpay simulator; integration tests for sign-in, profiles, listings and search.
- [ ] W4: Integration tests for request to pickup, duplicate payments, concurrent reservation and webhooks.
- [ ] W5: e2e full rental; overdue with the staging clock; monitoring services in Compose with Sasvat.
- [ ] W6: Security suite; sale and refund tests.
- [ ] W7: Performance suite with Locust; `staging` profile; demo seed data.
- [ ] W8: Resilience runs with Sasvat; full regression run; test report for the demo.

Interfaces

- **You need:**
  - endpoints from Rammohan and Siddharth as they land (W3 onwards)
  - monitoring configs from Sasvat (W5)
- **You give:**
  - the environment for everyone (W1)
  - the `test` profile for CI (W2)
  - the Razorpay simulator (W4)
  - cross-module suites
  - the week-1 research answers that unblock sign-in, payments and the app build

Done when:

- A fresh clone runs `make up && make seed` in under 10 minutes.
- Every PPT test exists and runs in CI.
- Demo data is seeded on staging.

Demo path, definition of done, decisions to confirm

Demo path

The final demo tells one story on two Android phones, a moderator's phone, and a laptop showing Jenkins and Grafana.

1. **Pipeline.** A PR runs through Jenkins: lint, tests, scans, image build, staging deploy, health check.
2. **Sign-in.** Student A signs in with Microsoft and completes onboarding.
3. **Listing.** A links a payout account and lists a camera for rent.
4. **Request.** Student B types "cam", picks the suggestion, sees the trust badge and late-fee rule, and requests 2 days. A approves, and B gets a push.
5. **Pickup.** Both upload photos, B pays by test UPI, and A enters B's OTP. The rental is Active.
6. **Overdue.** The staging clock jumps past the end time, the grace period and 2 more hours. The overdue banner shows the growing late fee, and A escalates.
7. **Return and dispute.** Both upload photos, then the OTP is exchanged. A records a damage charge, and B disputes it.
8. **Ruling.** The moderator rules on the appeal. B pays the late fee and the ruled charge, and A taps Proceed.
9. **Reviews.** Both review blind, the reviews reveal, and both trust scores update.
10. **Help.** B asks the chatbot about late fees, then hands off to a moderator.
11. **Operations.** Grafana shows the business and payments dashboards. The reconciliation report shows zero mismatches. The duplicate-payment test runs live.

Definition of done for every PR

- [ ] Lint and unit tests pass, and the touched code has tests.
- [ ] `contracts/` is updated if an API or event changed, and consumers have approved.
- [ ] Migrations run forward and back on a fresh database.
- [ ] No secrets, and no personal data in logs.
- [ ] Metrics are added if the change adds a flow listed in the metric table.
- [ ] Code owners of every touched folder have approved.

Decisions to confirm in week 1

Decision	Proposal	Who	By
Mobile framework	React Native with Expo, development build	Sripranav	W1
iOS	Demo on Android. iOS needs an Apple Developer account ($99 a year), although the product plan lists both platforms	Team	W1
Phone OTP provider	Firebase Phone Authentication	Siddharth, Parv	W1
Photo storage	MinIO in Compose and on staging	Parv	W1
Staging host	From Parv's research	Parv, Sasvat	W1
Container registry	GitHub Container Registry	Sasvat	W1
Staging clock for the demo	Admin-only clock offset, staging only	Rammohan	W5
Trust weights and risk-flag thresholds	As in Rammohan's section	Rammohan, team	W6
Razorpay fees in production	The platform takes no fee, so someone must cover the gateway's fee; not needed for the test-mode demo	Team	Before launch